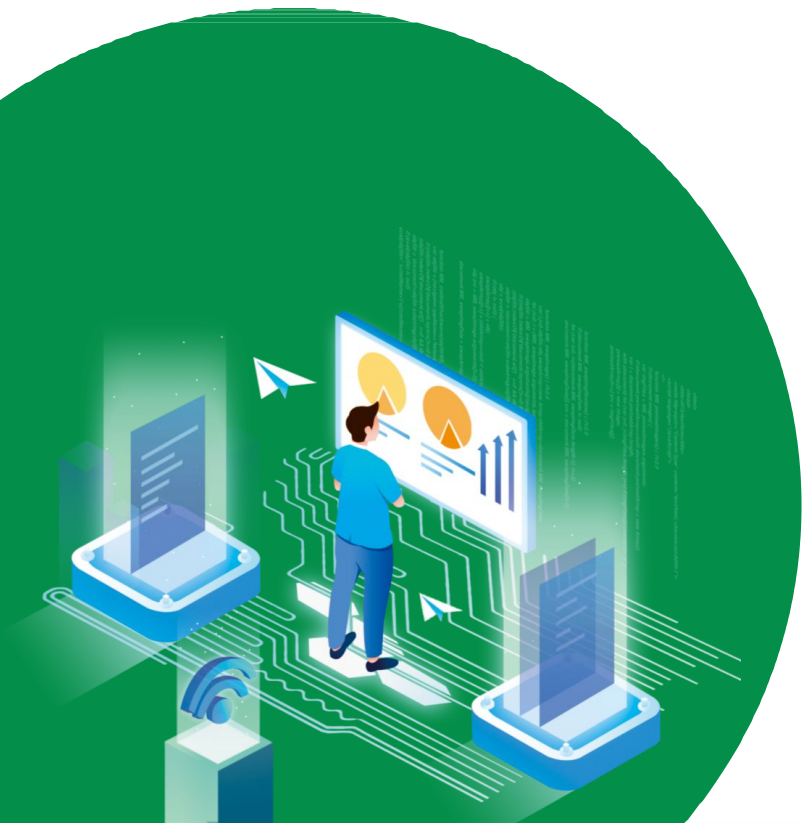


Minswap - Performance W1

Presenter: Link



CONTENT

01. Postgresql

02. Web, Sentry issues

1

Postgresql



Logging & Monitoring



Database statistic



Configuration optimization

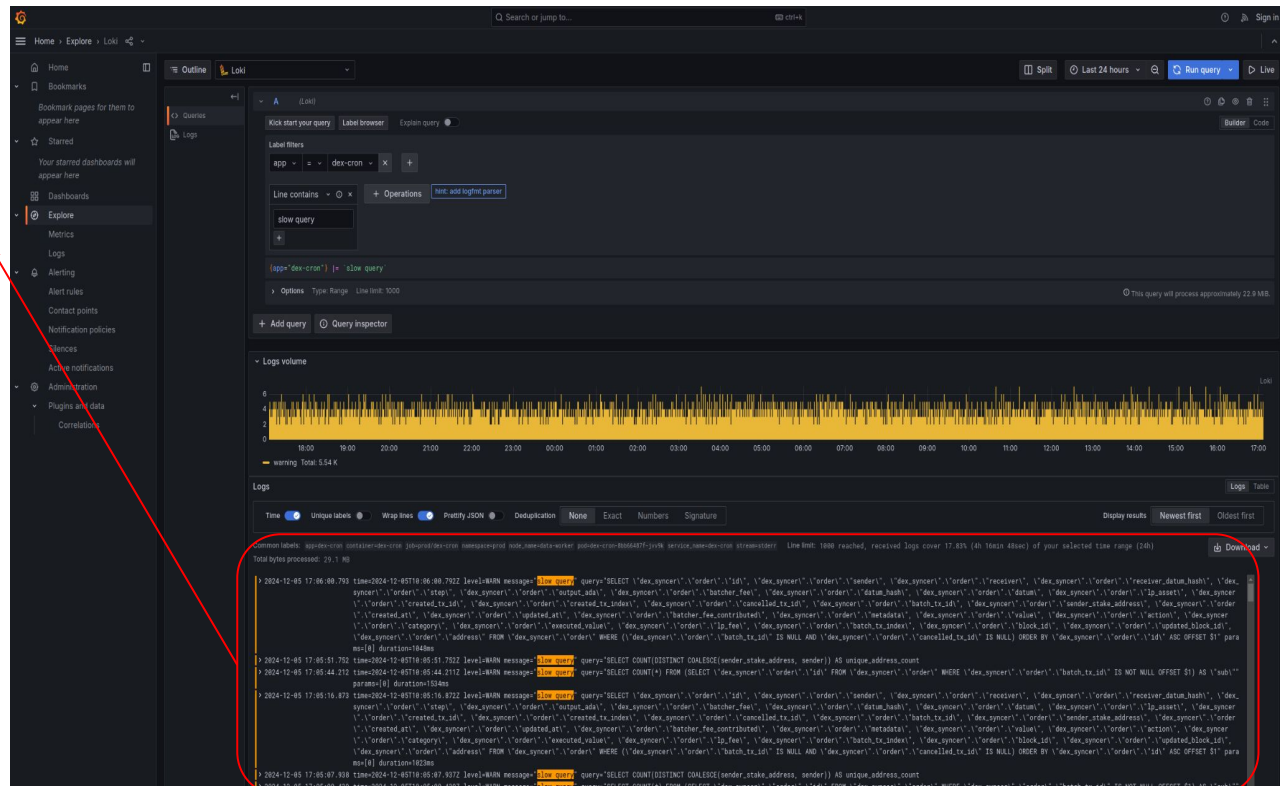


PG: Logging & Monitoring

Logging slow queries on application side:

- Can miss very big slow queries
- Should be json format and able to query, aggregate (count, sum, avg,...)

=> Should log on PG server side, json format (v16 and above)





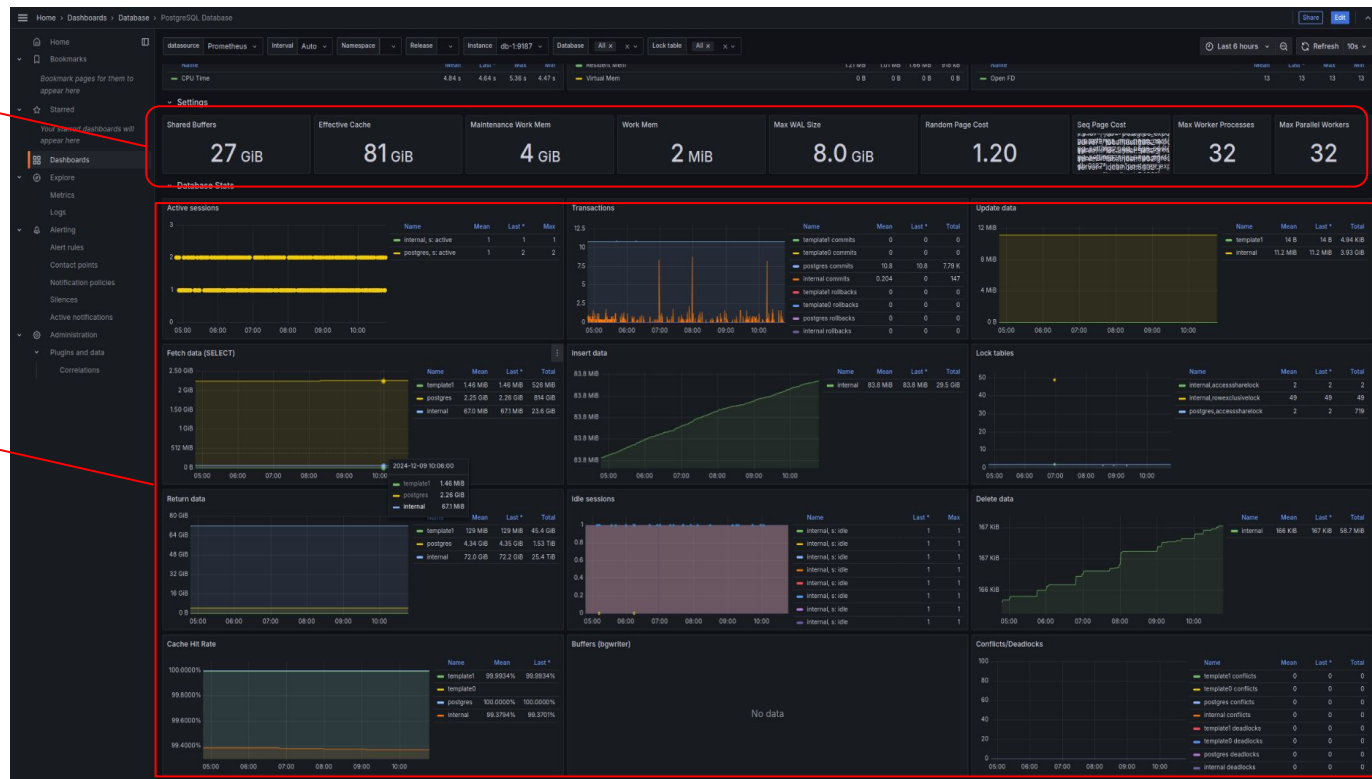
PG: Logging & Monitoring

Important configs

Most of default configurations are not optimize for a specific domain

Key Insights:

- Read heavy application
- Cache hit rate very high
- Fetch too many data to application side

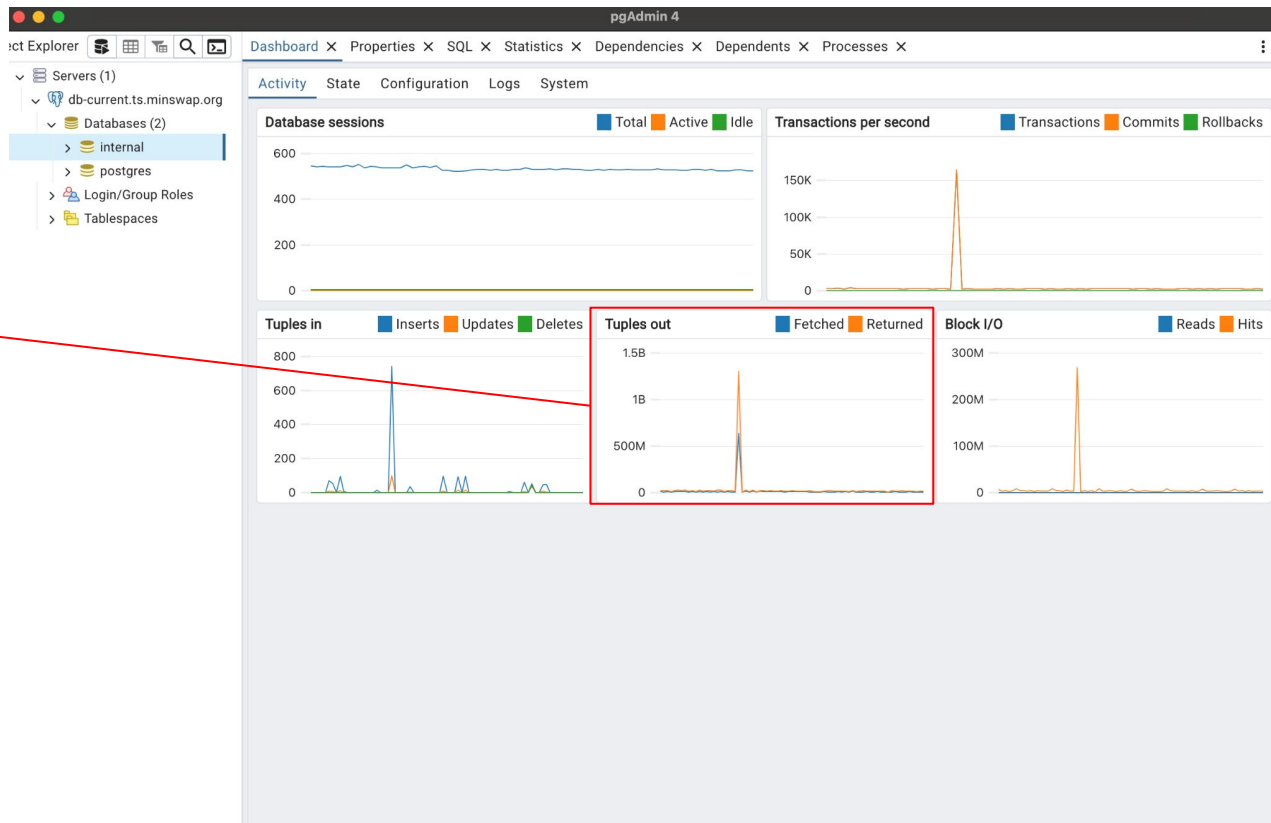




PG: Logging & Monitoring

PGAdmin 4 monitor:

- Abnormal events: Very big read heavy

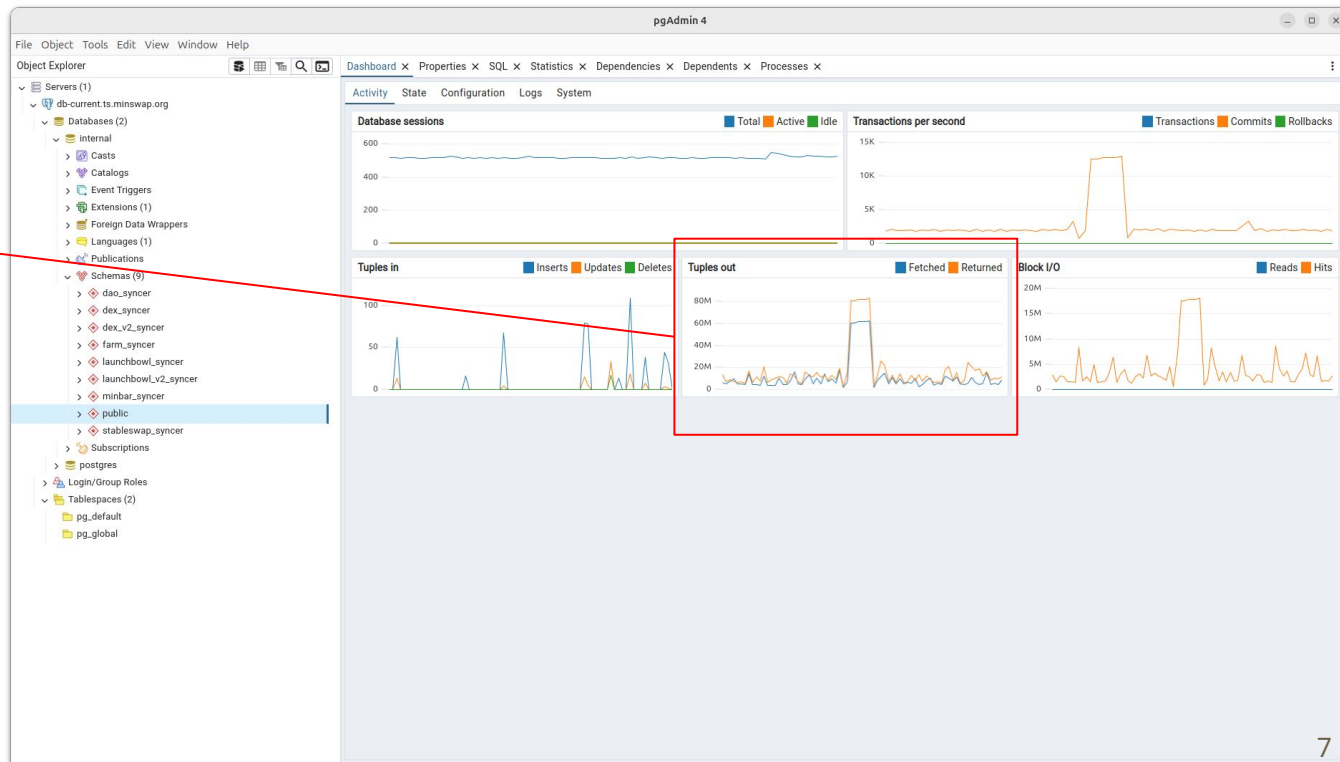




PG: Logging & Monitoring

PGAdmin 4 monitor:

- Abnormal events: Very big read heavy





PG: Logging & Monitoring

Object Explorer | Dashboard | Properties | SQL | **Statistics** | Dependencies | Dependents | Processes

Servers (1)
 db-current.ts.minswap.org
 Databases (2)
 > internal
 > postgres
 Login/Group Roles
 Tablespaces

Statistics	Value
Size	62.25 GB
Backends	533
Xact committed	437531469
Xact rolled back	1248
Blocks read	1488398206
Blocks hit	787173152095
Tuples returned	3301162653933
Tuples fetched	1489987200142
Tuples inserted	88351716
Tuples updated	11709941
Tuples deleted	217591

Db size & connections is normal

Queries hit cache 99%

Return too many data



PG: Optimize configurations

Importance file: *postgresql.conf*

- *shared_buffers*: Defines the memory PostgreSQL uses for its own buffer cache to store recently accessed data pages, index,...
 - 25-40% of RAM, balance with OS caching needs.
- *effective_cache_size*: Estimates how much memory is available for PostgreSQL to cache disk data (helps the PostgreSQL query planner decide whether to use indexes or sequential scans)
 - 50-75% of RAM to guide the query planner.
- *work_mem*: Sets the base maximum amount of memory to be used by a query operation (such as a sort or hash table) before writing to temporary disk files.
 - $\text{work_mem} = (\text{Total_RAM} * 0.02) / \text{concurrent_connections}$

```
root@testnet-preprod /etc/postgresql/14/main # ls
conf.d      pg_ctl.conf  pg_ident.conf  start.conf
environment pg_hba.conf  postgresql.conf

SELECT name, source, setting
FROM pg_settings
WHERE source != 'default'
AND source != 'override'
ORDER by 2, 1;
```

name	source	setting
effective_io_concurrency	configuration file	200
huge_pages	configuration file	try
lc_messages	configuration file	en_US.UTF-8
lc_monetary	configuration file	en_US.UTF-8
lc_numeric	configuration file	en_US.UTF-8
lc_time	configuration file	en_US.UTF-8
listen_addresses	configuration file	*
log_line_prefix	configuration file	%m [%p] %q%a%id
log_timezone	configuration file	Europe/Berlin
maintenance_work_mem	configuration file	4194304
max_connections	configuration file	7000
max_parallel_maintenance_workers	configuration file	4
max_parallel_workers	configuration file	32
max_parallel_workers_per_gather	configuration file	8
max_wal_size	configuration file	8192
max_worker_processes	configuration file	32
min_wal_size	configuration file	2048
port	configuration file	5432
random_page_cost	configuration file	1.2
shared_buffers	configuration file	3538944
ssl	configuration file	on
ssl_cert_file	configuration file	/etc/ssl/certs/ssl-cert-snakeoil.pem
ssl_key_file	configuration file	/etc/ssl/private/ssl-cert-snakeoil.key
work_mem	configuration file	2048
max_stack_depth	environment variable	2048



PG: Optimize configurations

Importance file: *postgresql.conf*

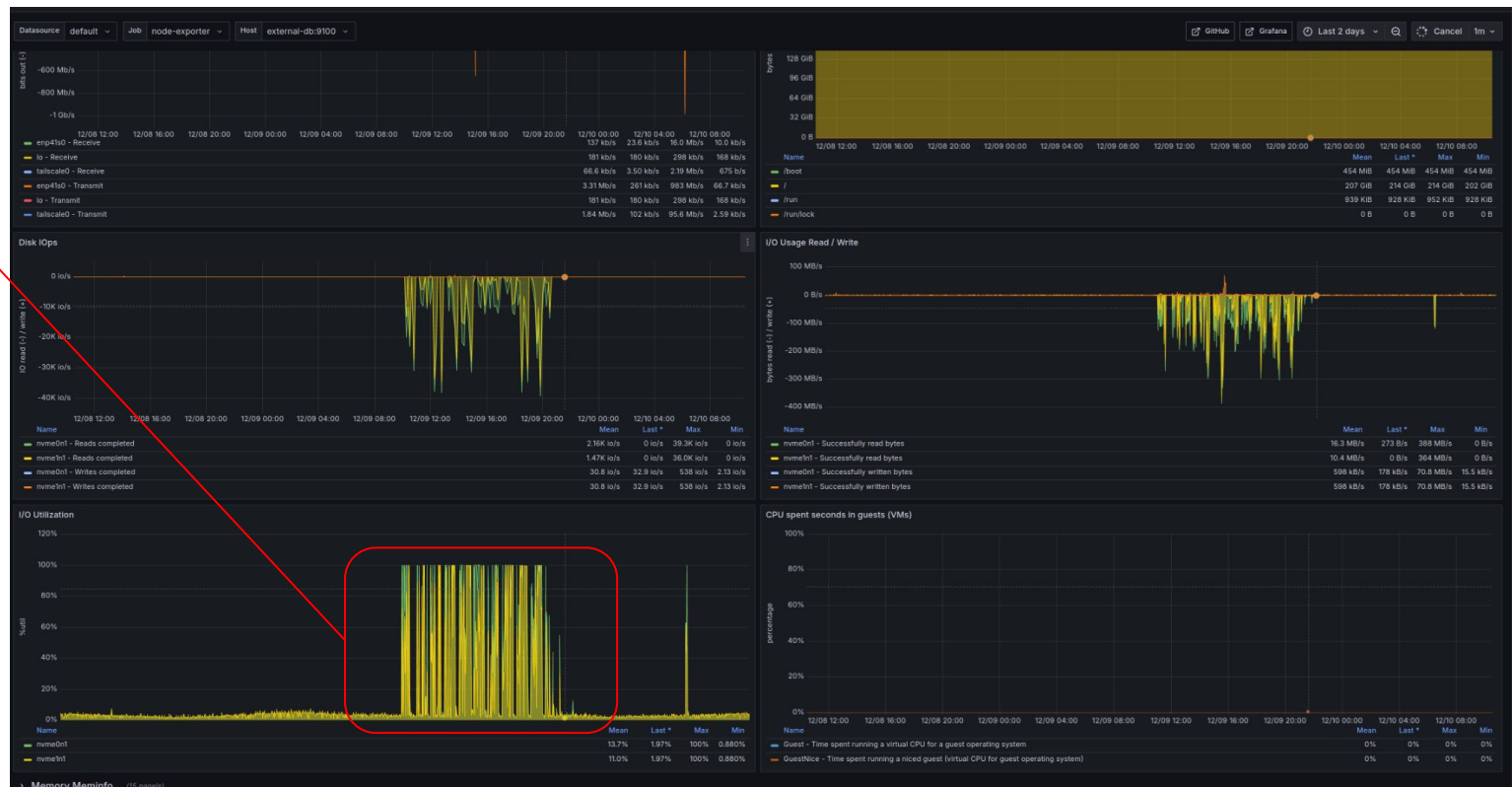
- ***maintenance_work_mem***: Memory allocated for maintenance operations like vacuuming, indexing, and adding foreign keys.
 - 1-2 GB for large databases with frequent maintenance operations.
 - Temporarily increasing the `maintenance_work_mem` configuration variable when loading large amounts of data can lead to improved performance. (Re-Syncer job)
- ***max_wal_size***: Sets the maximum size of Write-Ahead Logs (WAL) before triggering a checkpoint.
 - Higher values reduce the frequency of checkpoints, which improves write-heavy workload performance.
 - 2-16 GB for medium to large databases.
 - Temporarily increasing the `max_wal_size` configuration variable can also make large data loads faster. (Re-Syncer job)
- ***max_worker_processes***: Specifies the maximum number of background worker processes.
 - Includes parallel query workers, logical replication workers, and custom background workers.
- ***max_parallel_workers***: Sets the total number of parallel workers across all sessions.
 - Use approximately half the number of CPU cores for general-purpose systems. (16 cores cpu -> set = 8)



PG: External-db

Disk overload. Reasons:

- Hardware (lower memory)
- Slow queries





PG: Optimize configurations

Should increase when
re-insert data

- OLTP (transaction-heavy) workloads benefit more from higher **shared_buffers** and **work_mem**.
- OLAP (analytics-heavy) workloads may require higher **work_mem** and larger **effective_cache_size**

Setting	Purpose	Suggested Range	Minswap db-01
shared_buffers	Memory PostgreSQL uses for caching data pages.	25-40% of total RAM	27GB
effective_cache_size	Estimates how much memory is available for PostgreSQL to cache disk data	50-75% of total RAM	71GB
maintenance_work_mem	Memory for maintenance tasks	1-2 GB	1GB
max_wal_size	Maximum size of WAL before checkpoint	2-16 GB	8GB
work_mem	Memory per query operation	4-16 MB (higher if needed)	2-4MB
max_parallel_workers	Total parallel workers	Half the CPU cores	8



PG: Optimize configurations

Importance file: *postgresql.conf* (Logging)

- *logging_collector=on* # enable capturing of stderr and csvlog
- *log_min_duration_statement=1000* # capture slow queries more than 1s
- *log_statement='all'* # capture all queries
- *log_statement_sample_rate=1.0* # capture all queries
- *log_line_prefix = '%m [%p] %u@%d '*
- *log_destination='csvlog'* # should upgrade to pg v16 and use json format
- *log_timezone = 'Etc/UTC'* # should use UTC timezone
- *shared_preload_libraries = 'pg_stat_statements'* # add statistic statement into pg_catalog


PG: pg_catalog

pg_stat_activity: current queries on system

- Should separate services account, easy to tracing and monitoring
(Slow queries, dead lock, read/write io, index usage,...)

Check queries running:

WHERE state='active'

Check blocked queries:

AND wait_event IS NOT NULL

```
select dbname, username, client_addr, query, query_start, state from pg_stat_activity
```

Output postgres.pg_catalog.pg_stat_activity

	dbname	username	client_addr	query	query_start	state
317	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
318	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
319	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
320	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
327	internal	launchdool-v2-syner	enulls	<insufficient privileges>	enulls	enulls
328	postgres	readonly-dev	100.87.55.78	select dbname, username, cli	2024-10-05 07:17:45.019149 +00:00	active
329	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
330	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
331	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
332	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
333	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
334	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
335	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
336	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
337	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
338	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
339	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
340	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
341	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
342	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
343	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
344	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
345	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
346	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
347	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
348	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
349	internal	launchdool-syner	enulls	<insufficient privileges>	enulls	enulls
350	internal	readonly	enulls	<insufficient privileges>	enulls	enulls
351	internal	readonly	enulls	<insufficient privileges>	enulls	enulls



PG: pg_catalog

pg_lock: check transaction lock (write heavy)

- DML statements
- Row lock (insert, relation, update,...)
- Table locks (Alter table,...)

Most of lock are related to relation lock (foreign key)

-> Should ignore foreign key check in re-insert data job (Improve 40% insert performance)

```
SELECT locktype, database, relation, mode, granted
FROM pg_catalog.pg_locks
```

	locktype	database	relation	mode	granted
1	relation	8356892	8367729	AccessShareLock	• true
2	relation	8356892	8367718	AccessShareLock	• true
3	virtualxid	<null>	<null>	ExclusiveLock	• true
4	relation	13761	12290	AccessShareLock	• true
5	virtualxid	<null>	<null>	ExclusiveLock	• true
6	relation	8356892	8367729	AccessShareLock	• true
7	relation	8356892	8367718	AccessShareLock	• true
8	virtualxid	<null>	<null>	ExclusiveLock	• true
9	relation	8356892	8367729	AccessShareLock	• true
10	relation	8356892	8367718	AccessShareLock	• true
11	virtualxid	<null>	<null>	ExclusiveLock	• true
12	relation	8356892	8367729	AccessShareLock	• true
13	relation	8356892	8367718	AccessShareLock	• true
14	virtualxid	<null>	<null>	ExclusiveLock	• true
15	relation	8356892	8367729	AccessShareLock	• true
16	relation	8356892	8367718	AccessShareLock	• true
17	virtualxid	<null>	<null>	ExclusiveLock	• true



PG: pg_catalog

pg_stat_user_table: check read scan foreach table

pg_statio_user_table: check I/O foreach table

Return too many data

```
select * from pg_stat_user_tables order by seq_scan desc;
```

relid	schemaname	relname	seq_scan	seq_tup_read	idx_scan	idx_tup_fetch	n_tup_ins	n_tup_upd	n_tup_del	n_tup_hot_upd	n_live_tup
8367733	farm_syncer	harvest_history	7635281	953853598020	1954669	2499788	625625	625763	155	0	624992
8367210	dex_syncer	last_pool_state	4895156	17883757966	10270983	11825417	10713	3821988	638	217523	10875
8367339	dex_v2_syncer	trading	1328656	122886096550	4196619	17456271089	494123	0	2481	0	491704
8367385	dex_v2_syncer	invalid_order	559401	306357223	5028	10839	38644	3520	1213	0	790
8367407	dex_v2_syncer	whitelisted_batcher	463751	1391240	0	0	3	0	0	0	3
8367247	dex_syncer	trading	311789	177826004394	1211309	116927037858	3981190	0	700	0	3980453
8367646	dao_syncer	lp_fee_proposal	292491	0	84	0	0	0	0	0	0
8367495	stablesmap_syncer	pool	232069	2052627849	52880	18588629	25814	0	154	0	24073
8367718	farm_syncer	staking_history	213857	62401969736	7012350	610620991342	644480	595447	167	0	644208
8367821	minbar_syncer	staking_history	140558	2378423888	2902244	2947143	16929	9105	15	0	16652
8367356	dex_v2_syncer	last_pool_state	116830	192381844	1562465	2119528	3760	441157	2037	1479	2813
8367169	dex_syncer	duplicate_pool	87809	0	0	0	0	0	0	0	0
8367505	stablesmap_syncer	pool_asset	73077	1020964	2702	5096	26	0	0	0	14
8368015	launchbowl_v2_syncer	ido_event	64711	11072346	7494	328635	242	0	0	0	185
8367226	dex_syncer	invalid_order	54735	23932922	4794192	13007	12398	1987	376	0	1668



PG: pg_catalog

Unuse index:

```
select seq_scan - idx_scan as is_miss_idx, *
from pg_stat_all_tables where not schemaname in ('pg_catalog', 'information_schema', 'pg_toast') order by is_miss_idx desc;
```

Session Result 59 x

	is_miss_idx	reldid	schemaname	relname	seq_scan	seq_tup_read	idx_scan	idx_tup_fetch	n_tup_ins	n_tup_upd	n_tup_del	n_tup_hot_upd
1	5666828	8367733	farm_syncer	harvest_history	7621485	952115627960	1954585	2499568	625621	625759	155	0
2	554227	8367385	dex_v2_syncer	invalid_order	559255	308241883	5028	10839	38644	3514	1213	0
3	463667	8367407	dex_v2_syncer	whitelisted_batcher	463667	1390988	0	0	3	0	0	0
4	291788	8367646	dao_syncer	lp_fee_proposal	291872	0	84	0	0	0	0	0
5	178909	8367495	stableswap_syncer	pool	231769	2049885817	52860	18588585	25804	0	154	0
6	87788	8367169	dex_syncer	duplicate_pool	87788	0	0	0	0	0	0	0
7	70281	8367505	stableswap_syncer	pool_asset	72982	1019634	2701	5094	26	0	0	0
8	57137	8368815	launchbowl_v2_syncer	ido_event	64631	11057546	7494	328635	242	0	0	0
9	32481	8367898	launchbowl_syncer	csi_order	32481	0	0	0	0	0	0	0
10	25627	8368857	launchbowl_v2_syncer	ido_order	32082	153048450	6375	4794	4794	4794	0	0
11	21198	8367698	dao_syncer	proposal_option	21198	2691955	0	0	221	0	0	0
12	19967	8367672	dao_syncer	pool_fee_request	19967	0	0	0	0	0	0	0
13	19452	8367835	minbar_syncer	harvest_history	45467	289606278	26015	25142	6562	6565	9	0
14	11626	8367622	dao_syncer	vote_history	11630	178506834	4	4584	15358	0	0	0



PG: pg_catalog

Unuse index:

```
select * from pg_stat_user_indexes order by idx_scan;
```

```
select * from pg_stat_user_tables where relname = 'order';
```

Session postgres.pg_catalog.pg_stat_user_indexes

	relid	indexrelid	schemaname	relname	indexrelname	idx_scan	idx_tup_read	idx_tup_fetch
1	8367475	8367498	stableswap_syncer	order	stable_order_sender_idx	0	0	0
2	8367115	8367118	public	schema_migrations	schema_migrations_pkey	0	0	0
3	8367131	8367137	dex_syncer	transaction	transaction_pkey	0	0	0
4	8367131	8367139	dex_syncer	transaction	transaction_tx_id_key	0	0	0
5	8368015	8368023	launchowl_v2_syncer	ido_event	ido_event_created_tx_id_created_tx_index_key	0	0	0
6	8367143	8367149	dex_syncer	utxo	utxo_pkey	0	0	0
7	8367598	8367605	dao_syncer	proposal	proposal_block_id_idx	0	0	0
8	8367348	8367353	dex_v2_syncer	batch	batch_pkey	0	0	0
9	8367429	8367435	dex_v2_syncer	event	event_pkey	0	0	0
10	8367407	8367413	dex_v2_syncer	whitelisted_batcher	whitelisted_batcher_pkey	0	0	0
11	8367396	8367404	dex_v2_syncer	global_setting	dex_v2_global_setting_created_tx_id_idx	0	0	0
12	8367672	8367678	dao_syncer	pool_fee_request	pool_fee_request_pkey	0	0	0
13	8367672	8367688	dao_syncer	pool_fee_request	pool_fee_request_block_id_idx	0	0	0
14	8367385	8367391	dex_v2_syncer	invalid_order	invalid_order_pkey	0	0	0
15	8367451	8367457	stableswap_syncer	transaction	transaction_pkey	0	0	0
16	8367451	8367459	stableswap_syncer	transaction	transaction_tx_id_key	0	0	0
17	8367372	8367383	dex_v2_syncer	factory	dex_v2_factory_updated_block_id_idx	0	0	0
18	8367463	8367469	stableswap_syncer	utxo	utxo_pkey	0	0	0
19	8367372	8367378	dex_v2_syncer	factory	factory_pkey	0	0	0
20	8367364	8367369	dex_v2_syncer	datum_hash	datum_hash_pkey	0	0	0
21	8367475	8367491	stableswap_syncer	order	stable_order_receiver_idx	0	0	0
22	8367998	8367997	launchowl_v2_syncer	lbe_order	lbe_order_pkey	0	0	0
23	8367155	8367163	dex_syncer	pool	dex_pool_creat	0	0	0
24	8367976	8367983	launchowl_v2_syncer	lbe_seller	lbe_seller_pkey	0	0	0

262 rows



PG: pg_catalog

Need archive tables:

```
SELECT table_catalog, table_schema, table_name
,pg_relation_size(table_schema || '.' || table_name) as size
FROM information_schema.tables
WHERE table_schema NOT IN ('information_schema', 'pg_catalog')
ORDER BY size DESC
LIMIT 10;
```

Output Result 22 x

	table_catalog	table_schema	table_name	size
1	internal	dex_syncer	utxo	16692019200
2	internal	dex_syncer	order	9097879552
3	internal	dex_syncer	trading	2962776064
4	internal	dex_v2_syncer	utxo	2755829760
5	internal	dex_syncer	pool	2560106496
6	internal	dex_syncer	event	2515075072
7	internal	farm_syncer	utxo	1996439552
8	internal	dex_v2_syncer	order	1274683392
9	internal	dex_syncer	transaction	1133248512
10	internal	stablesyncer	utxo	1058234368



PG: pg_catalog

n_dead_tup and Autovacuum

Table and index bloat:

A table bloat if some maintenance processes can't be run properly. Due to MVCC, table will contain a lot of older versions of rows, if these versions can't be removed.

There have been cases where a one-row table has grown to several gigabytes in size.

Even after the old versions are deleted, the table stays large size, visible rows being located at the end of the table and preventing PostgreSQL from shrinking the file.

2

Other Issues



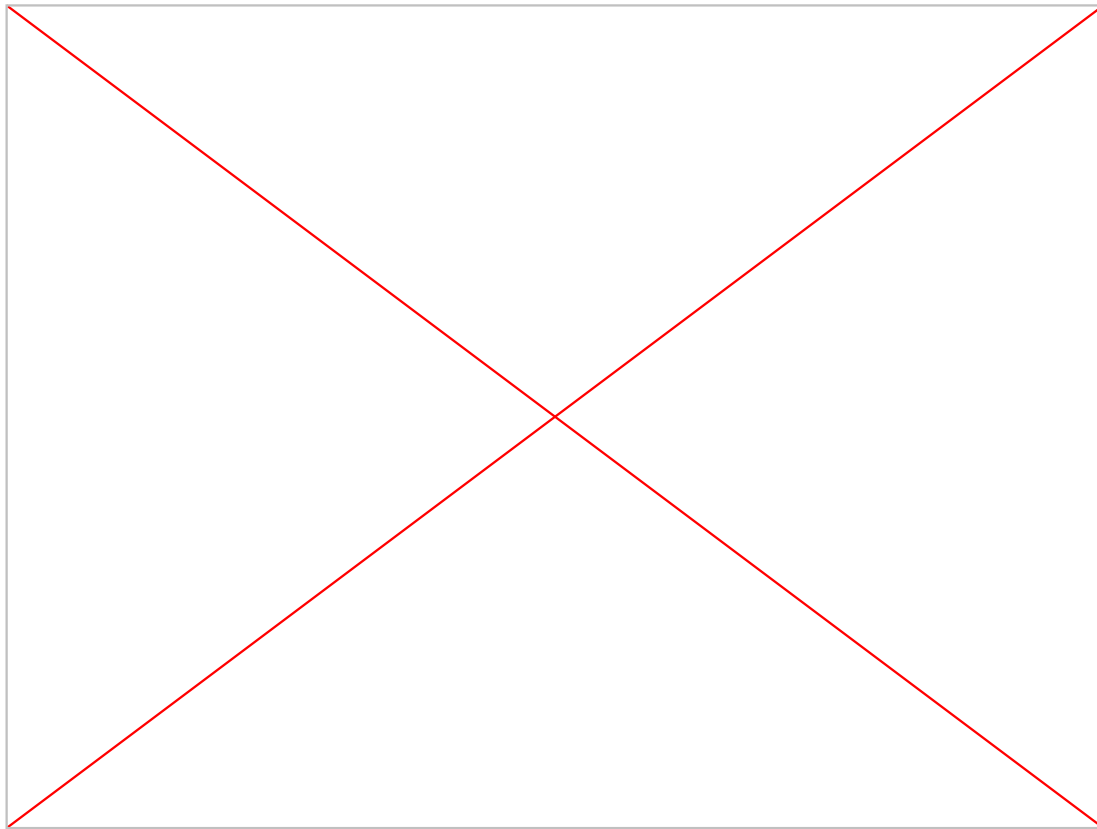
Web issues



Sentry issues



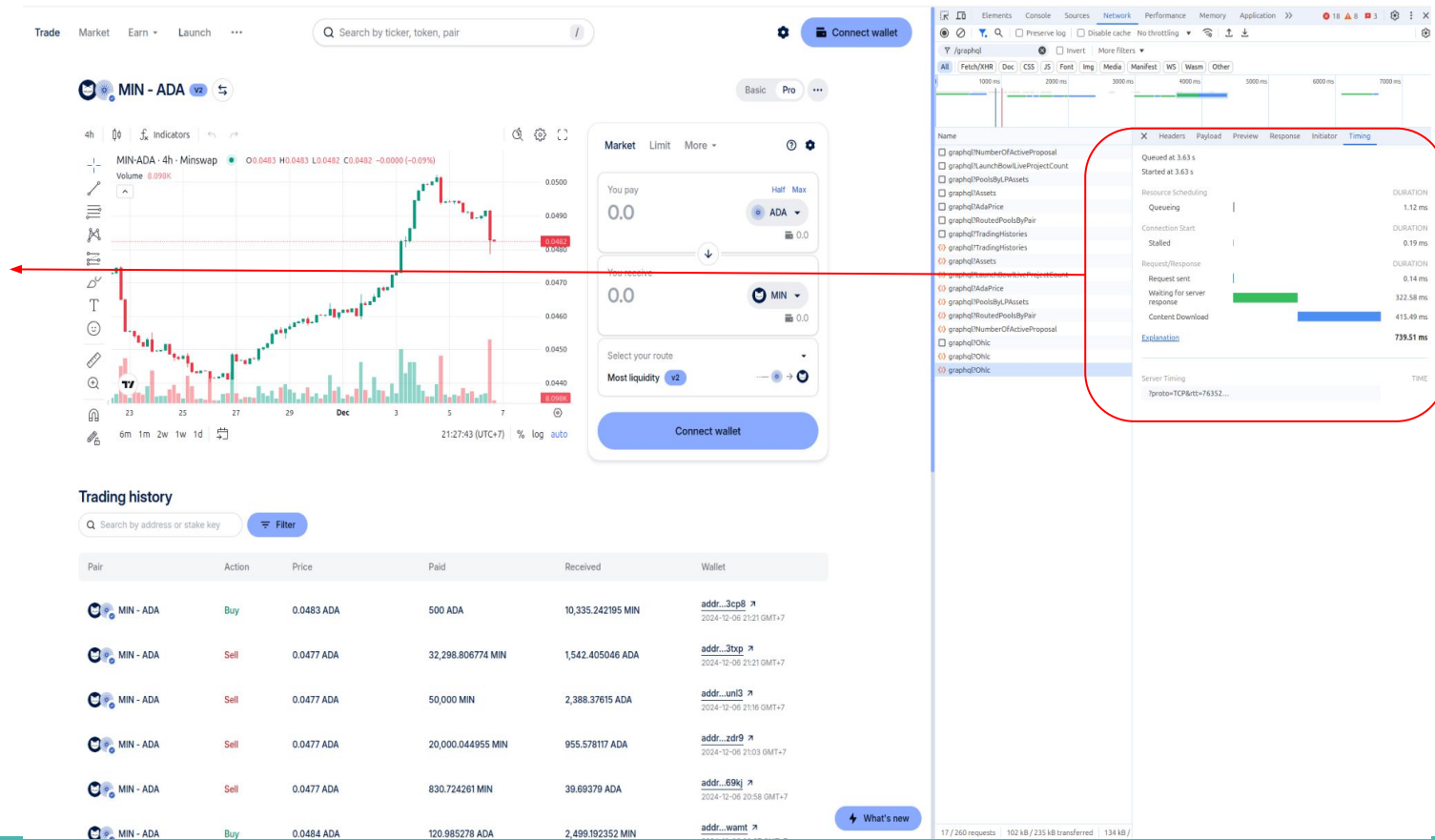
Web issues: ohlc API





Web issues: ohlc API

- Process start at **3.63s** -> bad user experience
- Waiting for server: **332ms**
- Content download: **415ms** (too large)





Web issues: ohlc API

Optimize guides:

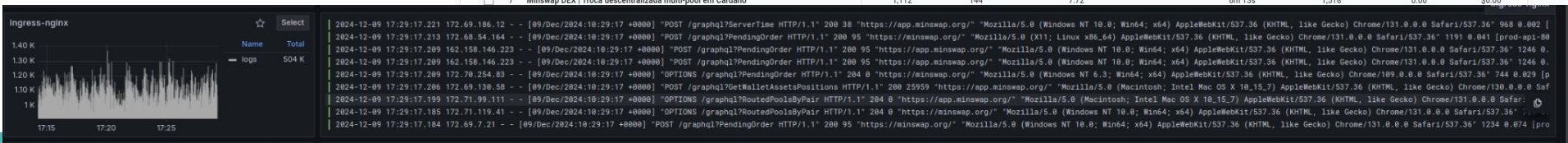
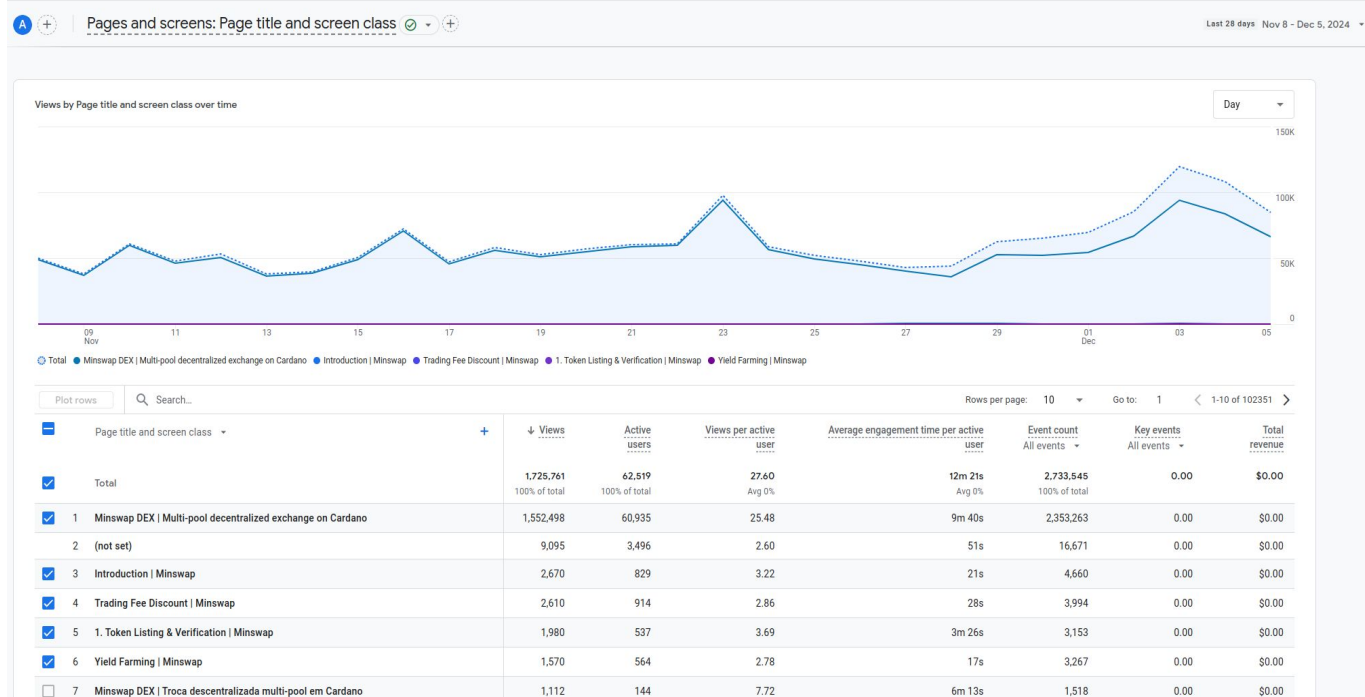
- **Frontend side:** Try to call *ohlc* and other APIs simultaneously -> Reduce 3s
- **Backend side:** Change a bit logic, optimize query and implement cache (for MIN-ADA,...) -> Reduce 30%
- **Devops:** On *nginx.conf*: Enable *gzip* option -> Reduce 50%
 - `gzip on;`
 - `gzip_disable "msie6";`
 - `gzip_vary on;`
 - `gzip_proxied any;`
 - `gzip_comp_level 5;`
 - `gzip_buffers 16 8k;`
 - `gzip_types text/plain text/css application/json application/x-javascript text/xml application/xml application/xml+rss text/javascript image/png image/gif image/jpeg;`



Web issues: Access log

Access log:

- Should have access log (count, sum, abnormal events,...)
- Should re-check GA config

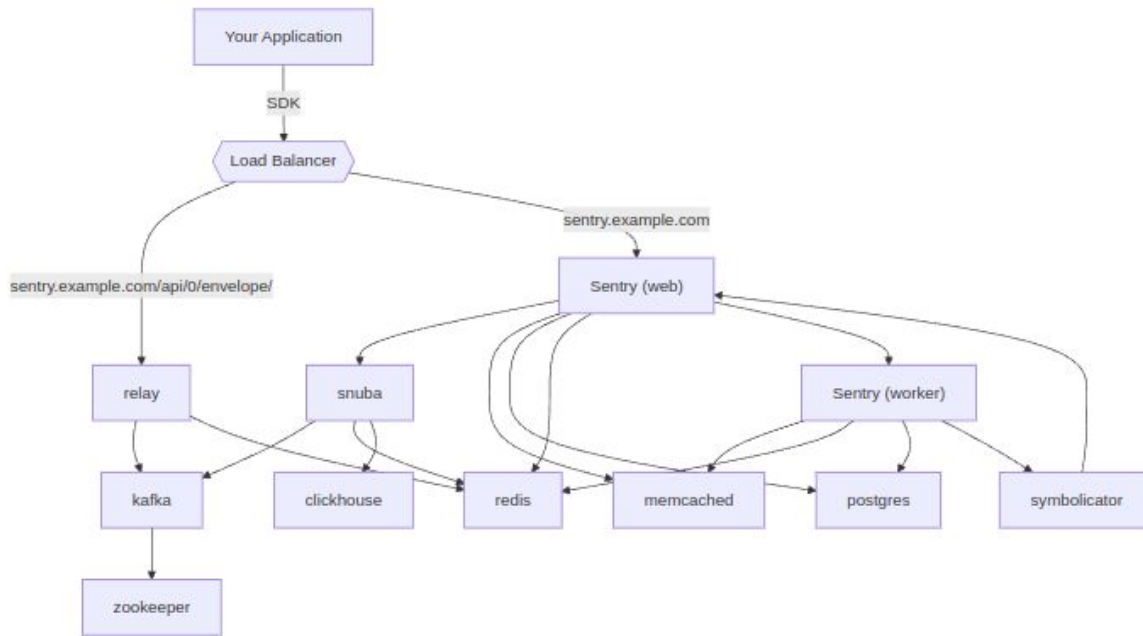




Sentry disk issues

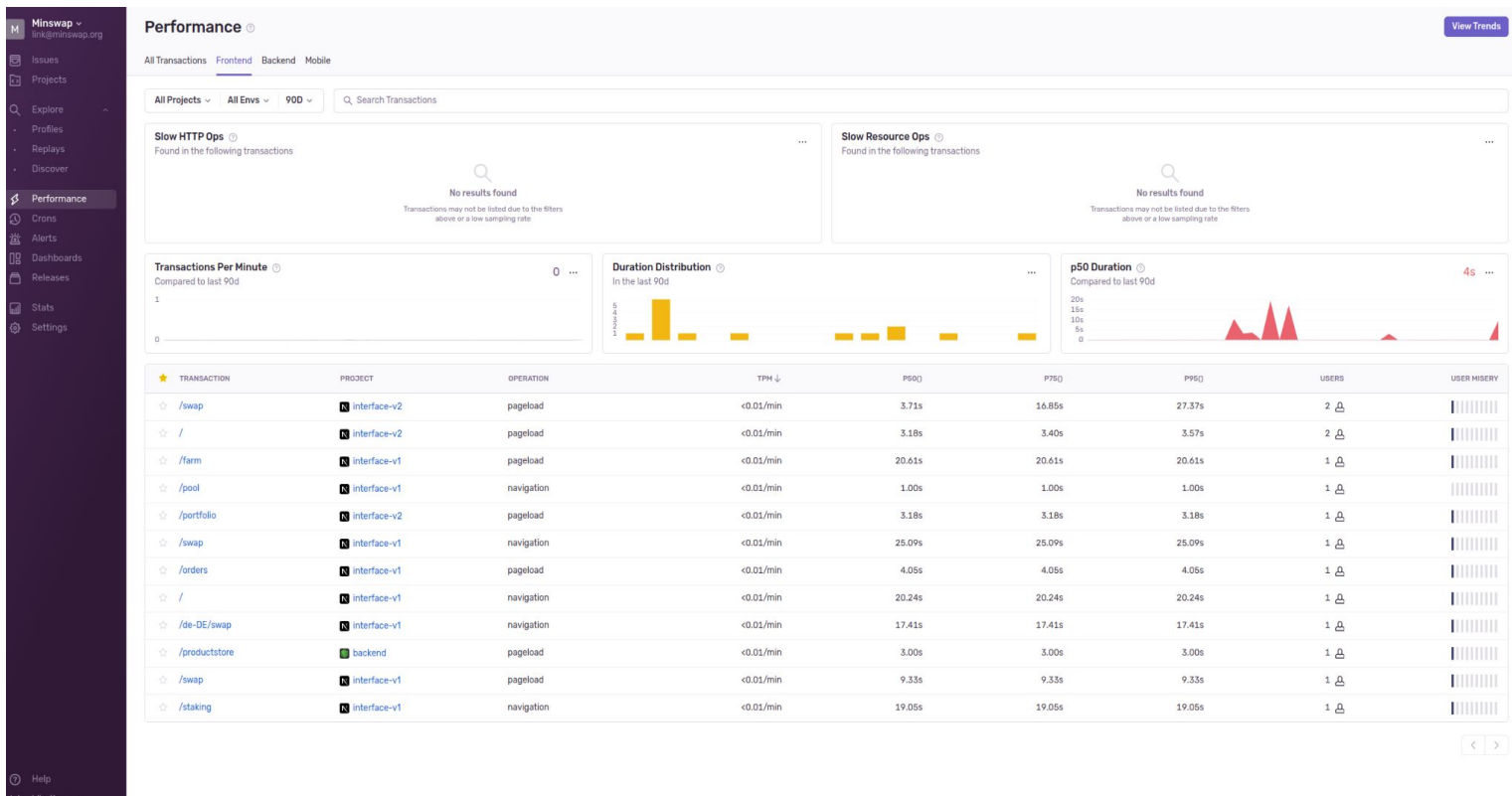
Understand Sentry architect first:

- *Error events*
- Performance monitoring events
- User experience (P99, P95, FCP, CLS, app crash,...)
- Crash, client capture events (Session replay,...)





Sentry disk issues





Sentry disk issues

- **Update application code:** Set `traceSampleRate` to 0.1. This configuration will reduce performance monitoring events from our application.
- **Update Sentry settings:** Configure *RateLimit* and *Inbound Filters* (only an admin can configure these). I've just checked, and these configurations are available on our Sentry version.
- **Create tasks to fix these issues:** The error issues on Sentry should be given High Priority because they affect our user experience.

The screenshot shows the Sentry Settings interface for a project named 'minswap'. The left sidebar contains a navigation menu with options like Issues, Projects, Explore, Profiles, Replays, Discover, Insights, Performance, User Feedback, Crons, Alerts, Dashboards, Releases, Stats, and Settings (which is currently selected). The main content area is titled 'Settings' and includes a breadcrumb trail: 'minswap > backend-m > Client Keys > Details'. The settings are organized into several sections: PROJECT (General Settings, Project Teams, Alert Settings, Tags & Context, Environments, Ownership Rules, Data Forwarding, User Feedback), PROCESSING (Inbound Filters, Security & Privacy, Issue Grouping, Debug Files, ProGuard, Source Maps, Performance, Replays), SDK SETUP (Client Keys (DSN), Loader Script, Releases, Security Headers), LEGACY INTEGRATIONS (Legacy Integrations), and CREDENTIALS. The 'Details' view for a 'Default' client key shows it is 'Enabled' and was 'Created' on 'Dec 4, 11:50 AM'. Under 'RATE LIMITS', there is a note about rate limits and a 'Rate Limit' configuration section with a 'Count' input field and a 'event(s) in' dropdown set to 'None'. The 'JAVASCRIPT LOADER SCRIPT' section contains a note about live changes and a script source: '<script src="https://sentry-2.minswap.org/js-sdk-loader/999808cf9f6ccdc8a362b0ef1621c482.min.js" crossorigin="anonymous"></script>'. The 'SDK Version' is set to '8.x'. Under 'Enable Performance Monitoring', 'Enable Session Replay', and 'Enable Debug Bundles & Logging', there are toggle switches, all of which are currently turned on. The 'CREDENTIALS' section at the bottom has a note about public and secret keys.



Summary

Next steps

- **Postgres:**
 - Setup backup database, separate service user, rewrite some config about logging, memory
 - Re-analysis slow query log (Count, sum, aggregate)
 - Adjust config(pg memories, worker, vacuum, foreign key,...) when run syncer job.
 - Learn about postgres optimize queries (long term)
- **Web:**
 - Check able to parallel call API from backend
 - Optimize slow queries
 - Adjust nginx config (include **gzip** option and **json access log**)
- **Sentry:**
 - Reduce traceSampleRate
 - Enable rate limit, Security options,...
 - Create task to fix issues and check daily, weekly



References

- PostgreSQL documentations: <https://www.postgresql.org/docs/> (Highly recommended)
- PostgreSQL Administration 9.6 cookbook (2017):
https://github.com/faisalbasra/postgres_books/blob/master/004-PostgreSQL%20%20Administration.Cookbook%209.59.6%20Edition.2017.4.pdf
- PostgreSQL 9.6 High performance (2017):
https://github.com/faisalbasra/postgres_books/blob/master/007-PostgreSQL.9.6.High.Performance.2017.5.pdf
- Sentry documentation: <https://docs.sentry.io/product/sentry-basics/>
- Web vitals: <https://web.dev/articles/vitals>